26088 Final Solution Architecture

Oct 11, 2026 · @Utkarsh

1. Problem statement

This is the final design for SIH problem statement 26088. It replaces the earlier architecture and web-app docs and fixes React as the front end.

Field	Value
Problem statement ID	26088
Title	Multilingual Cooperative Governance & Legal Assistance Chatbot
Organization and department	Ministry of Cooperation, National Council for Cooperative Training (NCCT)
Category and theme	Agriculture, FoodTech & Rural Development
Proposed mode	Software

The problem. Cooperative members, farmers and rural stakeholders often do not know cooperative laws, government schemes, PACS services, crop insurance, financial literacy basics or how to file a grievance, because of language barriers and limited access to reliable guidance.

The objective. Build an AI-powered multilingual chatbot that gives instant guidance on cooperative governance, legal provisions, schemes and member services.

Our interpretation and scope

- **Legal information, not legal advice.** The assistant explains what a public document says and where to complain. It never predicts the outcome of a dispute.
- **State-aware.** Cooperative law is layered between the central Multi-State Co-operative Societies Act and state Acts, so the assistant asks which state applies. We cover the central documents plus one chosen state.
- **Three languages.** Hindi, English and one regional language that a team member can evaluate, not all 22.
- **Depth on three topics.** Cooperative law and by-laws, crop insurance (PMFBY) and grievance guidance get the most depth. Schemes and financial literacy are lighter modules.

- **Software only, voice through the browser.** The microphone in the browser replaces the dedicated hardware terminal.

Requirement coverage. Every feature the statement lists maps to a page or capability in this design.

Required feature	How we cover it	Where
Multilingual conversational interface	Language selector, answers in the chosen language	Ask page, language service
Guidance on cooperative laws and by-laws	Retrieval over the central Act, Model By-laws and one state Act, with citations	Ask page, Law library
Ministry of Cooperation schemes and services	Structured scheme cards plus retrieval	Schemes and services page
PMFBY and agricultural support guidance	Guided crop-insurance flow with facts that carry a source and date	Crop insurance guide
Financial literacy assistance	Short lessons and an optional quiz	Financial literacy page
Cooperative grievance redressal support	Category, forum, steps, documents checklist and an editable draft	Grievance helper
Voice-enabled assistance for rural users	Browser recording, speech to text, text to speech	Ask page, speech service
Integration with mobile and web platforms	Responsive React web app that works in a phone browser	Whole front end
NLP, chatbot frameworks, speech, cloud	Retrieval-augmented generation, agent tools, Bhashini speech, containerized deployment	Backend services

How we will know it works. We score the system on an 80-question test set: retrieval hit rate, correct refusal of unanswerable questions, answer faithfulness to the cited source, speech recognition error rate, and response time. Targets are in the sprint section.

2. Proposed platform and solution

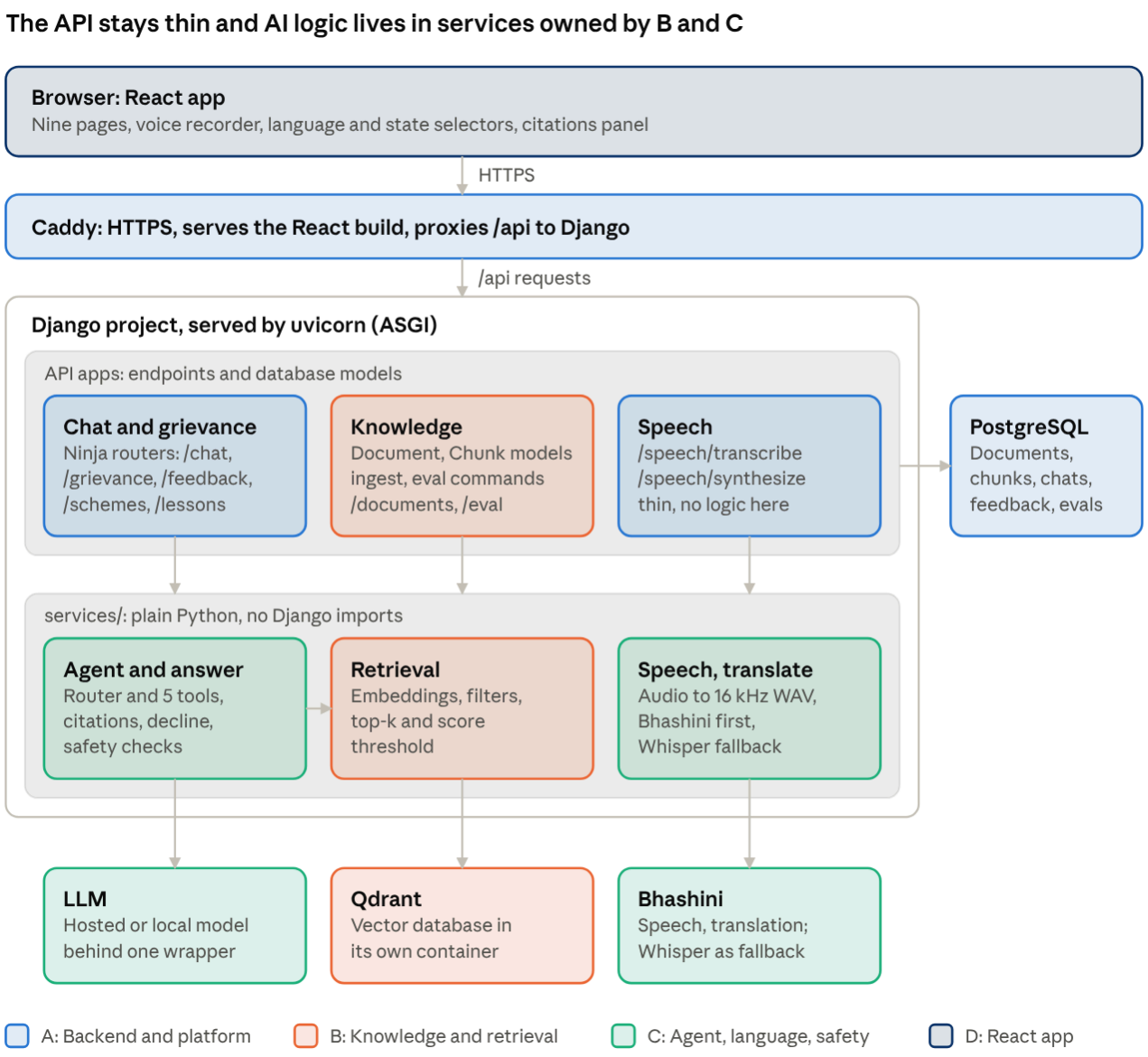
Sahkar Saathi (a working name) is a web platform where a cooperative member, farmer or PACS secretary asks a question by voice or text in their own language and gets a short answer that cites the exact source document, its date and the page. It combines retrieval

over a small, verified set of public cooperative documents with a few structured guides (crop insurance, schemes, grievances) and a speech layer.

What makes it different

- **Cited, dated answers.** Every answer shows its source, so a member or trainer can check it.
- **State-aware.** It asks which state applies before answering by-law questions, and filters retrieval by state.
- **Honest about limits.** Below a confidence threshold it says it does not know and names a helpline or office, rather than guessing.
- **Voice when it matters.** Members can speak, check the transcript, and listen to the answer.
- **Guided grievance help.** It turns a described problem into a forum, steps, a checklist and a draft, and never files anything itself.

Architecture

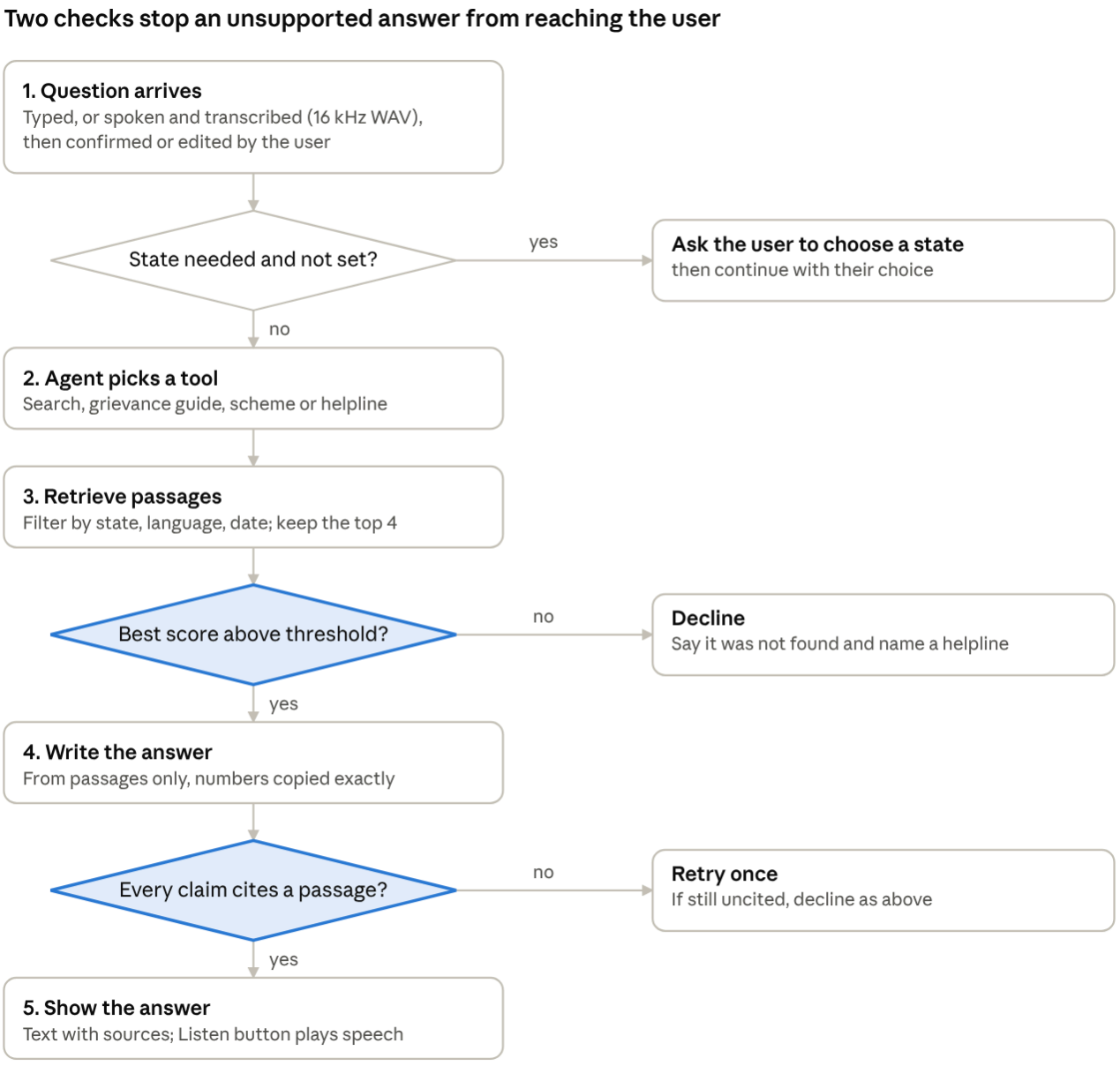


system architecture, coloured by owner

The browser reaches Django through Caddy, which serves the React build and handles HTTPS. Django's API apps own the endpoints and database models, and call plain-Python services for retrieval, the agent and speech. Colour shows the owner of each box, so this picture is also the ownership map for section 6.

How one question is answered

A question may arrive as speech, which is transcribed and confirmed first. The system asks for a state when the question needs one, retrieves filtered passages, declines if the evidence is weak, and otherwise writes an answer that must cite what it retrieved.

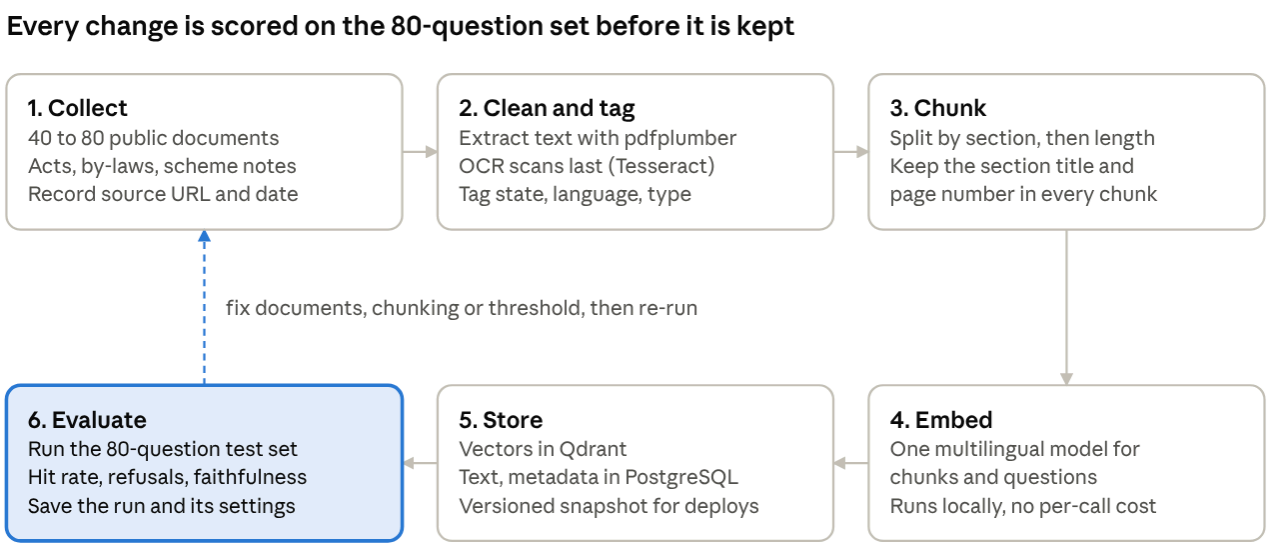


question flow, 5 steps and 3 checks

The two highlighted checks protect the user: a score check before writing, and a citation check after. Either failure leads to an honest decline with a helpline, never a guess.

How the knowledge base is built

Documents are collected once, cleaned and tagged, then chunked and embedded into Qdrant. The same 80-question test set scores every change, so tuning is measured, not guessed.



knowledge pipeline, 6 stages with a feedback loop

The pipeline runs left to right along the top row, then right to left along the bottom. The dashed loop is the habit that matters: when evaluation finds a gap, fix the documents, chunking or threshold, and run it again.

Core data

Table	Holds	Why it exists
Document	One row per source file: title, publisher, type, published date, state, language, version, status	Retire an outdated document without deleting it
Chunk	Passage text, page, section, state, language, vector id	Filtered retrieval and precise citations
Conversation and Message	Anonymous chat, with answer status, top score, model name and latency	Evaluation and debugging
Citation	Which chunks supported each answer	The citations panel
Feedback and GrievanceCase	Ratings and reasons; draft complaints that are never filed	Admin review queue
TestQuestion, EvalRun, EvalResult	The 80 questions, run settings and per-question results	Before and after evidence for the pitch

Stable facts such as premium shares, deadlines and helpline numbers live in `schemes.yaml`, `forums.yaml` and the lesson files, each with a source URL and date. The answer step quotes them directly instead of searching for them, which is more reliable.

3. Tech stack

Every tool is free and open source. Licenses I confirmed from sources carry a link; the others are from general knowledge, so check each license file when you install it. The backend keeps the earlier decision: Django with Django Ninja, if at least two members have built a Django project, otherwise FastAPI with the same structure.

Layer	Choice	License	Why and caveats
Front end	React with TypeScript, built by Vite	MIT, Apache 2.0, MIT	Fast dev server and a static build that Caddy can serve
Routing	React Router	MIT	One route per page
API calls	TanStack Query	MIT	Handles loading, errors, retries and caching
API types	openapi-typescript	MIT	Generates TypeScript types from the backend's `openapi.json`, so the front end cannot drift from the contract
UI translations	react-i18next	MIT	One strings file per language
Styling	Tailwind CSS	MIT	Quick responsive layout with large, readable text
Voice in the browser	`getUserMedia` and `MediaRecorder`, built into browsers	Free	Needs HTTPS or localhost, see below
Front-end tests	Vitest, and Playwright for the scripted conversations	MIT, Apache 2.0	Playwright can replay the five demo conversations before each gate
Backend	Django, Django Ninja, Pydantic, uvicorn	BSD, MIT, MIT, BSD	Typed API, admin site, async handlers

Layer	Choice	License	Why and caveats
Relational database	PostgreSQL	PostgreSQL License	Documents, chats, feedback, evaluation results
Vector database	Qdrant, self-hosted	Apache 2.0	The default Docker setup has no authentication, so keep it on the private Compose network
Embeddings	multilingual-e5-small or base, or BGE-M3 if memory allows	MIT	Run locally with sentence-transformers. BGE-M3 has 568M parameters and needs about 2.1 GB of memory. Compare on your test set
RAG framework	LlamaIndex or LangChain	MIT	Pick one
PDF extraction	pdfplumber or pypdf	MIT, BSD-3	Avoid PyMuPDF: its AGPL-3.0 license would force you to open-source the whole app or buy a license
OCR for scans	Tesseract with Hindi data	Apache 2.0	Only for scanned PDFs; do these last
Speech and translation	Bhashini through its API, with Whisper as an open-source fallback for speech to text	Free API registration; Whisper is MIT	Check Bhashini limits in week 1 and test the fallback per language
Audio conversion	ffmpeg, run as a separate process	Free software	Converts browser audio to the format the speech API expects
Language model	Chosen by the evaluation, see the risks and open decisions	Varies	Local open-weight model, free hosted tier, or generate in English and translate

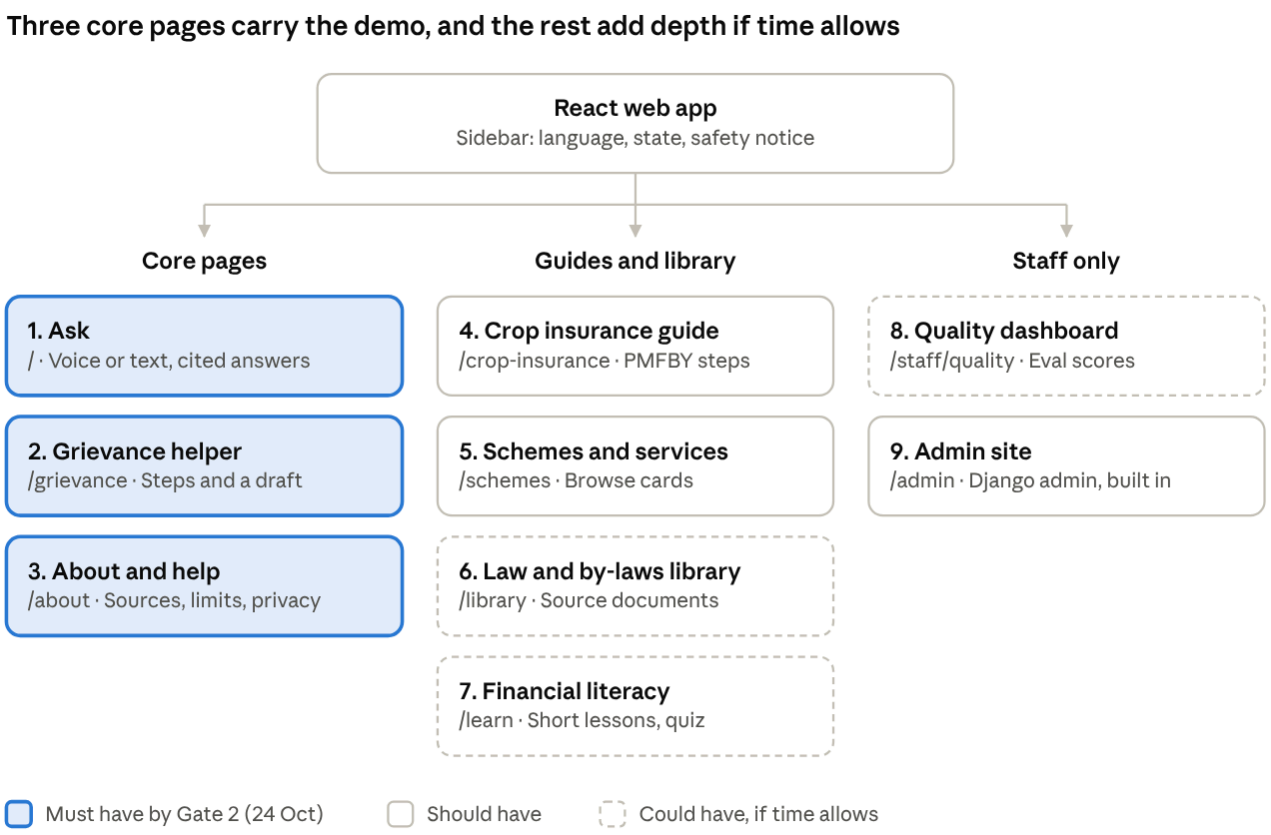
Layer	Choice	License	Why and caveats
Containers and proxy	Docker Compose, Caddy	Apache 2.0	Caddy serves the React build, proxies `/api` to Django on the same origin (no CORS setup), and provides HTTPS
Quality tooling	pytest, Ruff, ESLint, Prettier, GitHub Actions	MIT, free allowance	Lint and test on every pull request

Voice needs HTTPS or localhost. Browsers expose `getUserMedia` only in a secure context, meaning HTTPS or localhost. Opening your laptop's address from a phone over plain HTTP will silently disable the microphone. Present from `localhost`, or put Caddy in front with HTTPS (a free tunnel can give a phone a secure address).

Browser audio is not what the speech API wants. Browsers record in formats such as WebM, and you should detect support at runtime with `MediaRecorder.isTypeSupported()` (same guide). A Bhashini client library says 16 kHz mono WAV is the safest input, so the backend converts every upload to that format before calling it. Budget half a day for this and test it on a phone.

4. Pages and functions

The React app has nine pages: seven public and two for staff. Three are must-haves by Gate 2 on 24 October (Ask, Grievance helper, About), two are should-haves, and the rest are could-haves. A shared sidebar holds the language and state selectors, so every page uses the same choices.



sitemap, 9 pages in 3 groups

Start with Ask: the guides and library open it with a prefilled question, and the staff pages report on it.

Page and route	Main functions	API it calls	Priority and sprint
1. Ask `/`	Text box and microphone. Editable transcript before sending. Answer with a citations panel (title, date, page, link). Listen button. Follow-up suggestions. Thumbs up or down with a reason. Legal-information notice. A clear "could not find this" state with the right helpline	`/chat/ask`, `/speech/transcribe`, `/speech/synthesize`, `/feedback`	Must. Text in Sprint 1, voice in Sprint 2
2. Grievance helper `/grievance`	Describe the problem by text or voice. Detected category and forum. Step list and documents checklist. Editable draft, copy or download. Reminder that nothing is filed automatically	`/grievance/guide`	Must. API in Sprint 2, page in Sprint 3
3. About and help `/about`	How it works and what it cannot do. Sources with dates. Languages. Privacy note. Verified helpline numbers. Feedback form	`/documents`, `/feedback`, `/meta/*`	Must. Sprint 2

Page and route	Main functions	API it calls	Priority and sprint
4. Crop insurance guide `/crop-insurance`	Pick season and crop type, see the farmer's premium share, how and by when to report a loss, where to enrol, the helpline. Each fact shows source and date. "Ask a follow-up" opens Ask prefilled	`/schemes/pmfby`, `/chat/ask`	Should. Sprint 2
5. Schemes and services `/schemes`	Cards by category with search. Each says what it is, who can use it, how to access it, with source and date. "Ask about this" opens Ask prefilled	`/schemes`	Should. Sprint 2 to 3
6. Law and by-laws library `/library`	Filter by state, type and language. Table with title, publisher, date, version and a link to the original. "Ask about this document" limits the next question to it	`/documents`	Could. Sprint 3
7. Financial literacy `/learn`	Short lessons on loans and interest, KCC basics, savings and spotting frauds, in the chosen language. Optional three-question quiz	`/lessons`	Could. Sprint 3

Page and route	Main functions	API it calls	Priority and sprint
8. Quality dashboard `/staff/quality`	Latest evaluation run: hit rate, correct declines, faithfulness, failed questions, comparison of two runs. Protected by a staff token	`/eval/latest`	Could. Sprint 3, used in the pitch
9. Admin site `/admin`	Add or retire documents, inspect chunks, review thumbs-down feedback, view conversations and grievance drafts, manage test questions	Django admin, built in	Free from Sprint 1

API endpoints. All paths sit under `/api/v1/`. The Django Ninja schemas are the single source of truth, and `openapi.json` is frozen on Sprint 1, day 2.

Method and path	Purpose	Key fields
POST `/chat/ask`	Ask a question in text	In: conversation_id, text, language, state. Out: message_id, answer_text, status (answered, partial, declined), needs_state, citations, followups
POST `/speech/transcribe`	Speech to text	In: audio, language_hint. Out: text, language, confidence
POST `/speech/synthesize`	Text to speech	In: text, language. Out: audio
POST `/grievance/guide`	Guided complaint help	In: description, state. Out: category, forum, steps, documents_needed, draft_text
GET `/schemes`, `/schemes/{slug}`	Structured scheme cards, with `pmfby` for the crop guide	Out: title, summary, eligibility, how_to_access, source_url, as_of

Method and path	Purpose	Key fields
GET `/lessons`	Financial-literacy lessons	Out: title, body, quiz
GET `/documents`, `/documents/{id}`	Knowledge-base browser	Filters: state, type, language
POST `/feedback`	Rate an answer	In: message_id, rating, comment
GET `/meta/languages`, `/meta/states`	Options for the selectors	Out: codes and display names
GET `/eval/latest`	Evaluation metrics (staff token)	Out: run config, hit rate, correct-decline rate, faithfulness
GET `/health`	Deployment check	Out: status, version, vector store reachable

Conventions: every answer carries `status` and `citations`, and a declined answer has no citations and names a contact. Errors use one shape, `{ "code": "...", "message": "..." }`. Language codes are short ISO codes such as `hi`, `en` and `mr`. Public endpoints are rate-limited. Streaming answers are a Sprint 3 stretch goal.

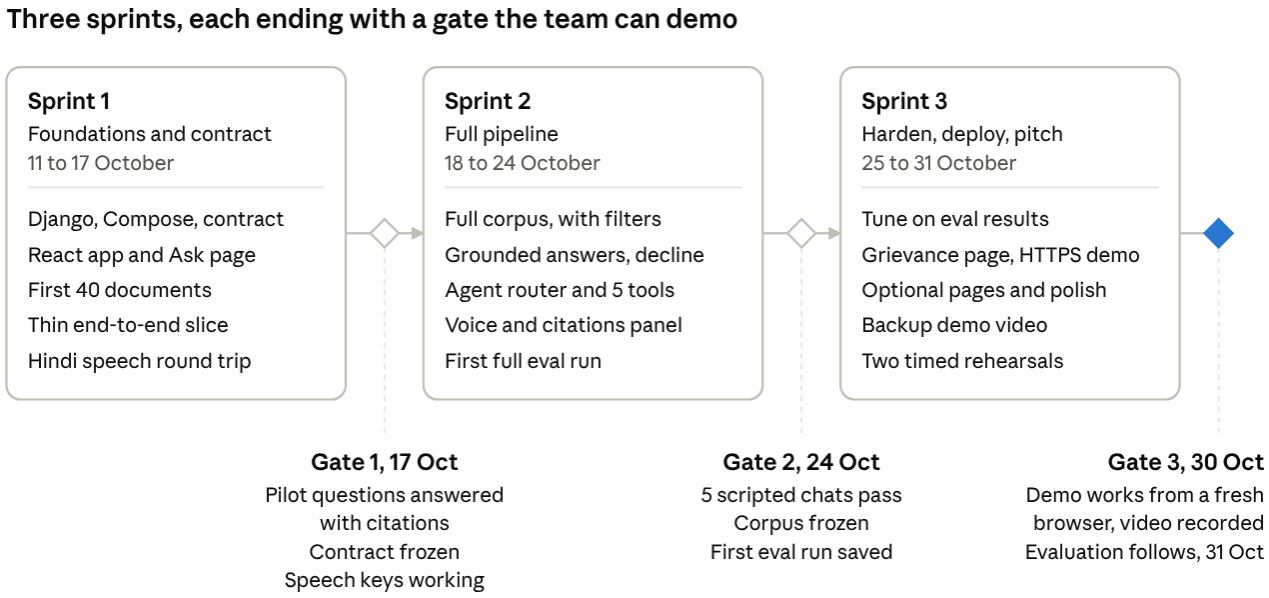
Shared front-end behavior

- **Voice flow:** a `useRecorder` hook records, sends audio to `/speech/transcribe`, shows an editable transcript, then sends the confirmed text to `/chat/ask`. If recording fails, the page falls back to typing.
- **Citations everywhere:** any answer shows its sources, with title, date and page.
- **Safety notice:** a fixed banner on Ask and Grievance helper says answers are information, not legal advice.
- **Accessibility:** large text, high contrast, short sentences, icons with labels, and layouts tested at phone width on a real phone.
- **Errors:** if the API is slow or down, show a clear message and the saved answers for the five scripted conversations (the `DEMO_CACHE` setting).
- **Privacy:** no login for the public, and no names, phone numbers or ID numbers stored.

5. Sprint overviews

The project runs as three one-week sprints, each ending with a gate that must work live. Sprint 1 builds the thinnest possible question-to-cited-answer slice, so integration problems appear in week 1. Sprint 2 builds the full pipeline and freezes the corpus. Sprint

3 is for tuning, deployment and rehearsal ahead of the evaluation on Saturday 31 October 2026.



three sprints with gates, not to scale

A gate is passed only when its criteria work live, not when the tasks are merely done.

Sprint	Goal	Main deliverables	Gate
Sprint 1 (11 to 17 Oct)	Foundations and contract	Django project and Compose setup. API contract frozen on day 2 with mock responses. React app skeleton with the Ask page in text mode. First 40 documents collected. Ingestion and retrieval working on 10 documents. First grounded answer with citations. Speech keys working and one Hindi speech round trip. 40 test questions drafted	**Gate 1, 17 Oct:** pilot questions answered with citations through the React app, contract frozen, speech keys working

Sprint	Goal	Main deliverables	Gate
Sprint 2 (18 to 24 Oct)	Full pipeline with all must-haves	Full corpus ingested with state, language and date filters. Grounded answers, decline logic and the ask-state flow. Agent router and five tools. Voice recording and playback with transcript confirmation. Citations panel, feedback, About page, Crop insurance guide, Schemes page. 80-question test set complete. First full evaluation run	**Gate 2, 24 Oct:** five scripted conversations pass end to end, corpus frozen, first evaluation run saved
Sprint 3 (25 to 31 Oct)	Harden, deploy, pitch	Tune thresholds, chunking and prompts using the evaluation. Grievance helper page. Law library, Financial literacy and Quality dashboard if time allows. Demo environment with HTTPS. Mobile and accessibility pass. Playwright replay of the scripted conversations. Backup demo video. Two timed rehearsals. Feature freeze on 28 Oct	**Gate 3, 30 Oct:** demo works from a fresh browser, backup video recorded, two rehearsals done

Targets for the final evaluation run. These are suggested goals for your team, not benchmarks.

Metric	Suggested target	How it is measured
Retrieval hit rate	85% or higher	80 questions with a known correct source; correct passage in the top 4 results
Answer faithfulness	90% or higher	Manual check of 30 answers: is every claim supported by the cited passage?

Metric	Suggested target	How it is measured
Correct refusal	90% or higher	20 out-of-scope or unanswerable questions: does the bot decline and redirect?
Speech recognition quality	Report the error rate honestly	20 recorded questions per language, transcript compared with what was said
End-to-end response time	Under 10 seconds	Voice in to voice out on a normal connection
User check	Report what confused people	5 people unfamiliar with the project ask 3 questions each in their language

If Gate 2 slips, cut scope in this order: streaming answers, Quality dashboard, Financial literacy, Law library, the second regional language. Never move the corpus freeze on 24 October, because Sprint 3 is the only buffer before the evaluation.

6. Work division

Each of the four members owns one layer from start to finish and has a clear deliverable in every sprint. Since the front end is now a full React app, the speech and translation services moved from the front-end member to the agent member compared with the earlier plan.

Member and ownership	Sprint 1 (11 to 17 Oct)	Sprint 2 (18 to 24 Oct)	Sprint 3 (25 to 31 Oct)
A. Backend and platform. Django, Ninja, database, Docker, Caddy, deployment, CI	Django project and Compose (web, Postgres, Qdrant, front-end build, Caddy). Ninja endpoints stubbed from the contract, `openapi.json` frozen, CI, admin for Document and Chunk	Wire `/chat/ask` to the services. Store Conversation, Message and Citation. Feedback, grievance, schemes, lessons and documents endpoints. Staff login, rate limiting, `/eval/latest`. Thin speech endpoints	Deploy the demo environment with HTTPS, health checks and logging. Async performance pass. Snapshot backup and restore. Final bug triage

Member and ownership	Sprint 1 (11 to 17 Oct)	Sprint 2 (18 to 24 Oct)	Sprint 3 (25 to 31 Oct)
B. Knowledge and retrieval. Corpus, ingestion, embeddings, vector store, evaluation	Collect and clean the first 40 documents. Loader and `ingest` command. Chunking v1. Embeddings and Qdrant collection. `VectorStore` interface. Draft 40 test questions	Ingest the full corpus with metadata. Retrieval with filters and scores. Finish the 80-question set. Eval runner and first full run. Corpus freeze on 24 Oct	Tune chunking, top-k and threshold using the evaluation. Record data sources and versions. Before and after results for the pitch. Verify every demo answer against its source
C. Agent, language and safety. Prompts, answer step, tools, guardrails, speech and translation services	LLM wrapper. Grounded prompt v1 with JSON output schema. First answer path on stub retrieval. `schemes.yaml`, `forums.yaml` and lesson content with sources. Bhashini keys and one Hindi speech round trip	Citation validation, decline logic, ask-state flow. Router and five tools. Safety module. Speech service with audio conversion to 16 kHz mono WAV, and translation. Manual faithfulness check on 30 answers	Prompt fixes from evaluation failures. Out-of-scope handling. Grievance guide polish. Test the fallback speech provider. Final safety tests. Prepare answers on guardrails for judges
D. React front end and pitch. Pages, voice interface, accessibility, demo video, slides	Vite, React and TypeScript app with routing. API client generated from `openapi.json`. Mock API. Sidebar with language and state. Ask page in text mode. Translation scaffold	Voice recording and playback with transcript confirmation. Citations panel. Feedback buttons. About page. Crop insurance guide. Schemes page. Second language strings	Grievance helper page. Law library, Financial literacy, Quality dashboard. Mobile and accessibility pass. Playwright run of the scripted conversations. Backup demo video, slide deck, rehearsal lead

Ownership by architecture layer

Layer	Owner
React app (all pages)	D

Layer	Owner
Caddy, Django project, API apps, PostgreSQL, deployment	A
Knowledge base, retrieval, evaluation	B
Agent, answer step, safety, speech and translation services	C
Test set	B consolidates; everyone writes 15 to 20 questions in their area

Shared duties

- Every pull request is reviewed by one other member within a day.
- Every service has a second person who can run it, so one absence does not stop the demo.
- Everyone rehearses the full pitch, so any member can run the demo.
- With three members, move C's speech duties to A, and cut the second language and the Quality dashboard first.

Hand-offs to watch: A to D (the frozen contract), B to C (the retrieval interface and score output), C to A (the answer and speech functions the endpoints call), and D to everyone (the transcript confirmation behavior).

Working agreements

- **Contract first:** schema changes go through a pull request announced in the team chat. A breaking change needs two approvals.
- **Mock first:** each service ships a stub in Sprint 1, so no one waits for another layer.
- **Branches:** `main` is protected, feature branches are short-lived, and everyone merges at least daily.
- **Definition of done:** merged, with a test or an evaluation question, running under Docker Compose, and noted in the README.
- **Settings, not constants:** model names, thresholds, top-k and prompt versions are settings, and each `EvalRun` records them.
- **Daily 15-minute sync,** blockers raised the same day, and a live demo of the gate criteria at each sprint end.
- **Freezes:** corpus on 24 October, features on 28 October, bug fixes only until the evaluation.

Risks and open decisions

Risk	Why it matters	Mitigation
Microphone blocked on a phone	Browsers allow recording only on HTTPS or localhost	Present from localhost, or serve through Caddy with HTTPS and test on a real phone in Sprint 2
Browser audio rejected by the speech API	The API works best with 16 kHz mono WAV	Convert on the backend, test with recordings from two phones and one laptop
Speech API limits or downtime	Voice is a headline feature	Request keys on day 1, keep a provider interface and a Whisper fallback, use `DEMO_CACHE`
Async and sync code mixed in Django	Sync-only parts fail inside async handlers	Keep AI logic in plain Python services, use async ORM methods, add an integration test of `/chat/ask`
Contract keeps changing	The React developer is blocked	Freeze on day 2, generate types from `openapi.json`, serve mocks
Confident wrong answers	Highest harm in this domain	Score threshold, citation validation, a decline path and a measured faithfulness check
Poor documents (scans, outdated files)	Weak retrieval	Cap at 40 to 80 documents, handle scans last, record version and date
Weak Indian-language output from a free model	Poor answers in Hindi or the regional language	Score two or three model options on the test set; fall back to generating in English and translating
Integration left to week 3	Late surprises	Thin end-to-end slice in Sprint 1 and a gate review each sprint
Demo fails live	Judges see a broken product	Present from one laptop, warm up `/health`, keep the backup video and cached answers

Sources

Drawn from web search results on 11 October 2026; I did not open each page in full. Licenses for tools not listed below (React, Vite, TypeScript, React Router, react-i18next, Tailwind CSS, openapi-typescript, Vitest, Playwright, Django, Django Ninja, uvicorn, PostgreSQL, LlamaIndex, LangChain, Tesseract, Whisper, ffmpeg, Docker, Caddy, ESLint, Prettier, pytest, Ruff) come from general knowledge, so confirm each before relying on it. Architecture, priorities, targets and sprint placement are my recommendations, not sourced facts. The problem statement text came from you.

- Recording audio in the browser (secure-context requirement, MediaRecorder format detection)
- Bhashini client SDK (16 kHz mono WAV as the safest speech input)
- TanStack Query licensing (MIT)
- Qdrant review (Apache 2.0, self-hosted)
- Qdrant overview (default Docker deployment is unauthenticated)
- BGE-M3 model listing (MIT, size, memory)
- multilingual-e5 comparison table (model sizes, MIT base models)
- pdfplumber vs PyMuPDF (MIT versus AGPL-3.0)
- Is PyMuPDF free for commercial use? (AGPL terms, pypdf BSD-3)
- Perspective: the MSCS (Amendment) Bill, 2023 (state legislatures regulate cooperatives under Part IX-B)